Skill Tracker

Full-stack application for tracking skills, practice sessions, and progress.

Tech Stack

Layer	Technology
Frontend	React + Vite (+ Nginx in local Docker)
Backend	Go (`net/http`)
Database	PostgreSQL
Local orchestration	Docker + Docker Compose
Cloud (recommended)	Vercel + Render + Neon

Project Structure

```
skill-tracker/
├── backend/
│   └── Dockerfile          # Go API image (used by Rende
├── cmd/server/             # API entrypoint
├── frontend/               # React + Vite (deploy to Ver
│   ├── Dockerfile          # Nginx image (local Compose
│   └── vercel.json
├── internal/
```

```
|   └── database/init.sql    # Schema (Compose init + appl
├── docker-compose.yml       # Local: db + backend + front
├── render.yaml              # Render Blueprint for the AP
└── README.md
```

Run Locally

Make sure you have Docker installed, then run:

```
docker compose up --build
```

Service	URL
Frontend	http://localhost:3000
Backend API	http://localhost:8080
Database	localhost:5432

Stop containers:

```
docker compose down
```

Reset database:

```
docker compose down -v
```

Local env (optional, without Compose)

Copy `.env.example` → `.env` (backend) and `frontend/.env.example` → `frontend/.env`.
With Vite alone, leave `VITE_API_URL` empty so the proxy in `vite.config.js` targets `localhost:8080`.

Database Schema

Schema file: `internal/database/init.sql`

`users`

Column
`id`
`nombre`
`color`
`creado_en`

`skills`

Column
`id`
`nombre`
`categoria`
`prioridad`
`estado`
`notas`
`ultima_practica`
`progreso`
`user_id`

`sesiones_practica`

Column
`id`
`skill_id`
`fecha`
`duracion_minutos`
`notas`
`progreso_percibido`

Cloud Deploy (Vercel + Render + Neon)

Target architecture:

```
Browser
  → Vercel (React static frontend)
      → Render (Go API, Docker Web Service)
          → Neon PostgreSQL
```

Deploy in this order so URLs and CORS line up.

1. Neon (database)

1. Create a Neon project and database.
2. Copy the connection string (include `sslmode=require`; prefer the **pooled** URL on the free tier).
3. In the Neon SQL Editor, run the contents of `internal/database/init.sql` (Compose does this automatically locally; Neon does not).

2. Render (backend API)

Blueprint: `render.yaml` (Docker Web Service, build context = repo

root, Dockerfile = `backend/Dockerfile`, health check = `/health`).

1. Connect the GitHub repo in Render and apply the Blueprint, **or** create a Web Service manually:

    - Runtime: **Docker**
    - Dockerfile path: `./backend/Dockerfile`
    - Docker build context: repository root (`.`)
    - Health check path: `/health`

2. Set environment variables in the Render dashboard (do **not** commit secrets):

Variable	Value
`DATABASE_URL`	Neon connection string
`CORS_ALLOW_ORIGIN`	Exact Vercel origin, e.g. `https://your-app.vercel.app`

    Do **not** set `PORT` — Render injects it.

3. Deploy and verify: `GET https://<your-service>.onrender.com/health` → `{"status":"ok"}`.

Notes:

- The API image includes CA certificates so TLS to Neon works from Alpine.
- On the free plan the service may sleep; the first request after idle can be slow.

3. Vercel (frontend)

1. Import the repo in Vercel.
2. Set **Root Directory** to `frontend`.
3. Framework: Vite — Build: `npm run build` — Output: `dist`.
4. Environment variable (**Production**, required at **build** time):

Variable	Value
`VITE_API_URL`	Public Render URL, e.g. `https://skill-tracker-api.onrender.com` (no trailing slash)

5. Deploy. SPA routing is handled by `frontend/vercel.json`.
6. If the Vercel URL differs from what you put in `CORS_ALLOW_ORIGIN`, update Render and redeploy the API (or restart so env is picked up).

Checklist

- ☐ Neon: schema applied (`init.sql`)
- ☐ Render: `DATABASE_URL` + `CORS_ALLOW_ORIGIN` set; `/health` OK
- ☐ Vercel: Root = `frontend`; `VITE_API_URL` set; rebuild after changing it
- ☐ Browser: create user / skill / session without CORS errors

---

Docker (local)

This project runs fully with Docker Compose:

- PostgreSQL with persistent volume `pgdata` and `init.sql`
- Backend on the internal Docker network
- Frontend served via Nginx after the Vite build (API proxied to `backend`)

Cloud production does **not** use the frontend Nginx container: Vercel serves the static build and calls Render directly via `VITE_API_URL`.

Notes

- Backend is stateless and configured via environment variables.
- There is no HTTP authentication; user selection is client-side (`localStorage`) — fine for a portfolio demo, not for private multi-tenant production.
- Designed for local Compose and for Vercel + Render + Neon with minimal code changes.

Author

Martin Lavin Carvajal
Skill Tracker — Full-stack learning project with Go + React + Docker